

Supplementary Material

for *Component Attribution in Agentic Knowledge Graph Question Answering: A Controlled Ablation, Budget Analysis, and Failure Census*

Md. Sakif Khan^a, Sadia Sharmin^a

^a*Department of Computer Science and Engineering, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh*

This file accompanies the manuscript named above. It carries four things the manuscript refers to but does not contain: the thirteen prompt templates the five systems send, the mechanics of the instrument at the width a reimplementations would need, how the two non-trivial baselines were built and how groundedness was measured, and five secondary measurements the manuscript reports no conclusion from. Section and table numbers prefixed S belong to this document; unprefixed ones refer to the manuscript.

S1. Prompt templates

Seven templates constitute the whole of AGR’s interface to the language model, and six serve the baselines: one for the parametric control, one shared by the two static retrieval baselines, and four belonging to the reimplemented agentic baseline. All thirteen are reproduced here verbatim from the source that sends them, in the order a question encounters them, by a script that reads the code (`scripts/build_paper_appendix.py` in the repository named in the manuscript’s data availability statement). No template carries a configuration constant, so one cached response serves every condition of a sweep; budgets are counters in code rather than instructions, and the evaluator’s remaining depth and backtracks are the only budget figures any prompt carries. Every call asks for JSON against a schema stated in the code beside the template.

S1.1. Planner

`agr/planner.py`, constant `PLANNER_PROMPT`. Sent once per question unless the planner is removed. The first exemplar is the single-hop case that is not decomposed. Braces written `{like this}` are format fields filled at send time; doubled braces are literal.

```
You decompose questions into a minimal ordered chain of sub-objectives for navigating a knowledge graph. Each sub-objective must be answerable by following one relation (possibly through an event node). Use #N to reference the result of sub-objective N. Use as FEW sub-objectives as possible.
```

```
For topic_mentions, extract ONLY specific named entities that appear literally in the question (people, places, organizations, works, products). Do NOT extract generic type words such as "celebrity", "university", "country", "senator", "airport", or "currency" -- these describe what kind of answer is wanted, not where to start searching.
```

```
Example 1 (single hop -- do NOT decompose):
Question: "who is the president of France?"
```

```

{"sub_objectives": ["find the president of France"],
 "topic_mentions": ["France"]}

Example 2 (composition):
Question: "who directed the film that won Best Picture in 1998?"
{"sub_objectives": ["find the film that won Best Picture in 1998",
                    "find the director of #1"],
 "topic_mentions": ["Best Picture"]}

Example 3 (conjunction):
Question: "which countries border France and have Spanish as official
language?"
{"sub_objectives": ["find countries that border France",
                    "find which of #1 have Spanish as an official language"],

 "topic_mentions": ["France", "Spanish"]}

Example 4 (superlative):
Question: "what is the largest city in the country where the Nile ends?"
{"sub_objectives": ["find the country where the Nile ends",
                    "find the largest city in #1"],
 "topic_mentions": ["Nile"]}

Question: "{question}"

```

S1.2. Relation scorer

agr/scorer.py, constant SCORE_PROMPT. One batched call per explorer pass over the twenty candidates with the highest embedding score. Not sent when $\alpha = 1$. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```

Sub-objective: {objective}

Candidate knowledge-graph edges (anchor entity --relation-->):
{candidates}

For EACH candidate, rate from 0.0 to 1.0 how likely following that edge from
that anchor leads toward completing the sub-objective. 1.0 = directly
answers it, 0.0 = irrelevant. Judge the RELATION MEANING in context of the
anchor, not surface word overlap.

```

S1.3. Evaluator

agr/nodes.py, constant EVAL_PROMPT. One call per explorer pass. The remaining depth and backtracks are substituted at send time; no other prompt carries a budget figure. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```

Current sub-objective: {objective}
Recently retrieved facts:
{facts}
Remaining budget: {d_left} more expansions, {b_left} more backtracks.

Decide:

```

- "answer" if the facts complete the sub-objective (list satisfying entities in "resolved", exactly as named in the facts),
- "continue" if this direction is promising but incomplete,
- "backtrack" if this direction is a dead end.

S1.4. Drafter and claim decomposer

agr/nodes.py, constant DRAFT_PROMPT. One call when the search ends. Produces the prose, the answer entities, and the atomic claims the verifier checks. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Facts retrieved from the knowledge graph:
{facts}
Entities identified as promising during exploration:
{candidates}

1. Draft an answer using ONLY the facts and candidates above. Use entity
  names exactly as written. If insufficient, say what could not be
  determined.
2. List the answer entities themselves in "answer_entities", copied EXACTLY
  as they are named in the facts above -- these are the entities your draft
  asserts as the answer, not every entity it mentions. If your draft is a
  hedge ("could not be determined"), "answer_entities" MUST be [].
  IMPORTANT: "answer_entities" must be specific entities of the kind the
  question asks for (a person for "who", a place for "where", etc.).
  Never restate the question or name entity types/categories as the
  answer. If you cannot name specific answer entities from the facts,
  your draft must be a hedge and "answer_entities" must be [].
3. Decompose your draft into atomic factual claims, each as
  {{ "h": entity name, "r": short relation phrase, "t": entity name }}.
  Only include claims your draft actually asserts. A hedged draft has zero
  claims.
```

S1.5. Entailment check

agr/nodes.py, constant ENTAIL_PROMPT. One batched call over the claims neither structural route settled. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Retrieved knowledge-graph triples:
{facts}

For each claim below, answer whether it is directly supported by the triples
above (true) or not (false). Unsupported includes: plausible but absent,
contradicted, or only inferable via outside knowledge.
Claims:
{claims}
```

S1.6. Rewriter

agr/nodes.py, constant REWRITE_PROMPT. One call on the give-up path only. The entity list it is given is already filtered; its own opinion about entities is discarded. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```

Question: {question}
Draft answer: {draft}
The following claims in the draft are NOT supported by the knowledge graph:
{unsupported}
The following entities remain verified and MUST be kept in the answer,
named exactly as written: {kept}

Rewrite the draft: keep the verified entities and their supported claims,
and remove or explicitly hedge the unsupported claims. If the kept list is
empty, state that the answer could not be verified against the knowledge
graph. Respond with prose only.

```

S1.7. Answerer (verifier skipped)

agr/nodes.py, constant ANSWER_PROMPT. Reached only when the drafting call itself failed; no committed run took this path. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```

Question: {question}
Facts retrieved from the knowledge graph:
{facts}

Entities identified as promising during exploration:
{candidates}

Answer using ONLY these facts and candidates. Give the answer entity name(s)
exactly as they appear above. If the information is insufficient, state what
could not be determined.
Also list in "answer_entities" the entities your answer
asserts (exactly as named above); [] if it asserts nothing.

```

S1.8. No-retrieval control

agr/baselines/noretrieval.py, constant PROMPT. The single call of the parametric control. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```

Answer this question with the name(s) of the answer entity or
entities. Be concise. If you do not know, say so and return [].

Question: {question}

```

S1.9. Vector-RAG and static GraphRAG

agr/baselines/vectorrag.py, constant PROMPT. Shared by the two static retrieval baselines, which differ only in what they retrieve. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```

Facts (retrieved from a knowledge graph):
{facts}

Question: {question}
Answer using ONLY the facts above. Name entities exactly as written in the
facts. If the facts are insufficient, say so and return [].

```

S1.10. Think-on-Graph: relation pruning

agr/baselines/tog.py, constant REL_PRUNE. One call per frontier entity per depth, over the first 40 relations returned. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Current entity: {entity}
Available relations (relation, direction, fanout):
{relations}
Select up to {w} relations most likely to lead to the answer.
Return {"relations": [{"rel": "...", "dir": "in|out"}]}
```

S1.11. Think-on-Graph: entity pruning

agr/baselines/tog.py, constant ENT_PRUNE. One call per selected relation per depth, over the first 20 neighbours returned. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Candidate entities reached (via {entity} --{rel}-->):
{entities}
Select up to {w} entities most likely on the path to the answer.
Return {"entities": ["..."]}
```

S1.12. Think-on-Graph: sufficiency

agr/baselines/tog.py, constant SUFFICIENT. One call per depth, on the last sixty gathered triples. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Knowledge triples gathered so far:
{triples}
Can the question be answered from these triples alone?
Return {"sufficient": true|false}
```

S1.13. Think-on-Graph: answer

agr/baselines/tog.py, constant ANSWER. One call when the search stops, whether it finished or hit the cap. Braces written {like this} are format fields filled at send time; doubled braces are literal.

```
Question: {question}
Knowledge triples:
{triples}
Answer using ONLY these triples; name entities exactly as written.
If insufficient, say so and return [].
```

S2. The instrument in detail

Section 3 gives the loop at the width the attribution study needs. This appendix records the mechanics a reimplementer would want, in the order a question meets them.

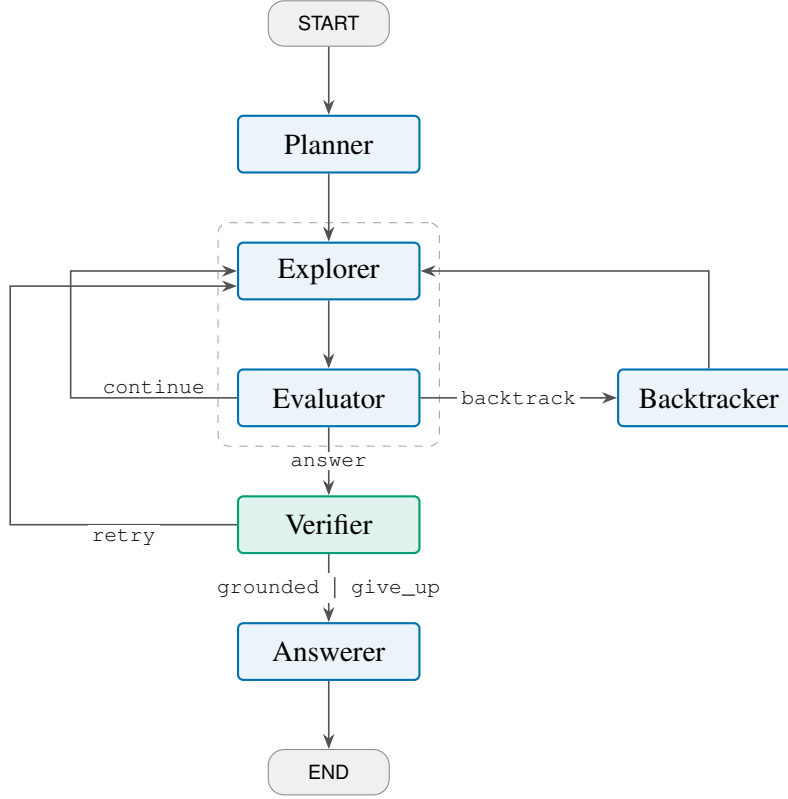


Figure S1: The AGR state machine. The dashed region is the Explorer $\leftrightarrow$ Evaluator search loop. Evaluator $\rightarrow$ Backtracker $\rightarrow$ Explorer restores an earlier frontier, and Verifier $\rightarrow$ Explorer is verification-driven re-exploration. The budgets of S2.5 bound all three cycles. Every path from Evaluator to Answerer passes through the Verifier.

S2.1. Graph tool API

Four graph operations back the loop, as parameterised Cypher templates against a Neo4j property graph rather than free-form queries the model would write itself, because a generated-query failure is indistinguishable in the results from a reasoning failure. `search_entity` resolves a surface form to a node through three tiers, exact name, then full-text index, then vector similarity, and stops at the first that returns a match. `get_relations` returns the relation types incident on an entity with their direction and fanout, after a blacklist removes schema relations, sorted by descending fanout and cut at 300 rows. `get_neighbors` returns at most 200 neighbours along one relation. Unnamed mediator nodes, which encode n -ary facts and make up 63.7% of the graph (Section 4.1), are traversed transparently, so a mediator path costs one hop. `verify_connection` asks whether two entities are adjacent directly or through one mediator, ignoring the relation. A fifth operation, `verify_triple`, was built first to check a claim $\langle h, r, t \rangle$ by asking whether that exact relation held between the two entities. It did not survive contact with generated text, because claim decomposition phrases relations in English (“played for”, “is the capital of”) and those phrases rarely correspond to a single Freebase predicate, so matching on them rejected claims that were true. Nothing in the running loop calls it.

S2.2. Edge scoring

The explorer gathers every relation row of every anchor into one candidate set, at most five anchors, and scores it in one pass. Each row c , an anchor together with a relation and a direction, receives

$$\text{score}(c) = \alpha \cos(v_{\text{rel}(c)}, v_o) + (1 - \alpha) \text{LLM}(c), \quad (1)$$

which blends two signals at the frozen $\alpha = 0.7$. The first is the cosine similarity between the candidate relation’s embedding and the embedding of the active sub-objective o , both from all-MiniLM-L6-v2 [1], with the relation vocabulary of 7,058 types embedded once in advance. The objective, not the original question, is what gets embedded, and that substitution is the concrete mechanism through which decomposition is meant to help. The second is a batched model judgement, one call, over the twenty candidates with the highest embedding score; every other candidate receives zero for that term. The blend is applied after the model call rather than inside the prompt, so that a prompt free of any configuration value lets an α sweep reuse one set of cached responses across every condition. The three best rows per anchor, skipping banned ones, are expanded, and the five neighbours with the highest reaching score become the next frontier.

S2.3. Evaluation, routing, and backtracking

The evaluator is one model call. It sees the active sub-objective, the thirty traversed triples most similar to it by the same embedding, and the remaining depth and backtrack budget, and it returns one of three decisions together with the entities it considers to satisfy the objective. Those entities are accepted only if they are names in the traversed set, so the evaluator cannot introduce an entity the graph did not return. When it marks an objective done, its resolved entities become the anchors for the next objective.

The router after it applies three tests in order. If the depth or the wall-clock budget is spent, the loop ends and the verifier runs. Otherwise, if a backtrack trigger fires and the backtrack budget allows, the backtracker runs; three triggers exist, namely that the last expansion added no triple, that the best score of the last expansion fell below $\tau = 0.20$, or that the evaluator asked to backtrack. Otherwise the evaluator’s decision stands. The backtracker pops the snapshot with the highest recorded score from the stack, which is often not the most recent one, restores its anchors and depth, and bans the anchor–relation–direction rows the most recent expansion followed. The mismatch between the snapshot it restores and the rows it bans is deliberate in neither direction, and Section 6.2 measures what it costs. This is where the mechanics part from Plan-on-Graph [2], whose decomposition and reflection live in prompts the model executes: here the plan is a data structure the program advances, two of the three backtrack triggers are deterministic tests on the scores, and the ban list, not a reflection, is what makes a restored frontier explore differently.

S2.4. Claim routing and the verifier’s outcomes

One model call sees the question, the sixty traversed triples most similar to it, and up to twenty-five candidates. It drafts prose, extracts the entities the draft asserts as its answer, and decomposes the draft into atomic claims over entity *names*, since the model has never seen a graph identifier; the relation slot is left as free text for the reason S2.1 gives.

Each claim is routed through up to three tests and leaves at the first that settles it. **Traversed adjacency** collapses the traversed triples into an undirected set of endpoint pairs; a claim is supported if its pair is in that set, and every traversed triple joining the pair is attached as citable evidence. This is the only route of the three that attaches evidence. **verify_connection** catches

claims whose endpoints were both reached but never walked together, querying the full graph for the same relation-agnostic adjacency. Claims that clear neither route, together with those naming an entity the agent never traversed at all, fall to an **entailment check**: one batched model call judging each remaining claim against the same fact window the drafter saw. Symbols decide, and the model is consulted only where the graph is silent.

The router that follows has three outcomes. No unsupported claim routes to grounded, and the draft is emitted verbatim with no further model call. An unsupported claim with verification, depth, and time budget remaining routes to retry: the verifier appends a repair objective naming the first unsupported claim, anchors the frontier on that claim’s head entity, and control returns to the Explorer. Otherwise the router takes *give_up*, and one rewrite call restates the draft around a *deterministically filtered* entity list, in which an entity survives if it appears in a supported claim or in no unsupported claim. An earlier design let the model regenerate the list itself, and that reintroduced exactly the ungroundedness the layer exists to prevent.

On the 80-question development set used to validate this design, 77 questions asserted no unsupported claim and took the grounded path at zero additional model cost. The remaining 3 took *give_up*, and the retry route did not fire once, because every question that reached the verifier with an unsupported claim had already exhausted its depth budget. On most questions, then, the layer is a pass-through that certifies a draft rather than a mechanism that visibly intervenes, which is the reading Section 6.2 confirms at test scale.

S2.5. Budgets and termination

Routers and the model client enforce every cap; no prompt requests one. A run may take at most 4 expansion depths, a beam of 3 candidate rows per anchor, 5 anchors in the frontier, 3 backtracks, 2 verification iterations, 300 seconds of wall-clock time, and 25 model calls. The depth, wall-clock, and verification caps are checked at two separate sites each, because the retry edge re-enters the Explorer without passing back through the router that would otherwise catch it. Every cap is hashed into the run configuration, so a reported result can always be traced to the caps that produced it. The two tool-layer caps of S2.1, 300 candidate relations and 200 neighbours, bound what the agent can see rather than what it can spend, and Section 4.2 shows why they matter for the comparison. Every cycle in Figure S1 therefore passes through a router that checks or decrements a monotone counter, so no sequence of model outputs, however adversarial, can produce a non-terminating run.

S2.6. Removing a component

Each ablation condition of Section 6 flips one flag and leaves every other line of the loop as it is. **No planner** replaces the planning call with a plan of one objective, the question itself; anchors still come from the annotated topic entities, as they do in the full system. **Embedding-only scoring** sets $\alpha = 1$, which makes no scoring call and ranks candidates by cosine similarity alone. **No backtracking** sets the backtrack budget to zero, so the router ends the search where it would have backtracked. **No verification** leaves the drafting call unchanged, entity list and claim decomposition included, but checks no route, makes no entailment call, and emits the draft verbatim. That condition therefore measures the checking and not the drafting, and its token saving is the cost of the routes rather than of the draft. The budget configuration is hashed into every record, so the condition that produced a row of Table 3 can be recovered from the row.

S3. Protocol detail

Section 4 states what each baseline licenses and how the comparison is controlled. This appendix records how the two non-trivial baselines were built and how groundedness was measured.

S3.1. *The static graph baseline*

Static GraphRAG [3] fetches a fixed-radius neighbourhood around the linked topic entities in one shot, reranks it by embedding similarity to the question, and answers from the top 30 facts. Two properties of that implementation bound what it licenses, and we state them rather than leave a reader to infer them from its scores. The radius is *one logical hop*: mediators are traversed transparently, so a mediator path costs two graph edges while remaining one hop, and a genuine two-hop chain is outside its reach by construction. And expansion retrieves at most 100 edges per topic entity with no ordering criterion, so on a high-degree entity the retained edges are an arbitrary sample rather than the best or the first; at least one topic entity is truncated on 72.5% of the 800 test questions. The baseline therefore licenses one narrow claim, the one Section 4.2 draws, and not a claim about what a two-hop static retriever would score. That experiment was not run.

S3.2. *The agentic baseline*

Think-on-Graph [4] performs model-guided beam search over relations and entities, reimplemented on this environment’s tools rather than quoted. Beam width and depth follow the original paper, 3 and 3. At each depth it makes one relation-pruning call per frontier entity, one entity-pruning call per selected relation, and one sufficiency call, so a full depth at width 3 costs up to thirteen calls, and the 25-call cap of S2.5 admits one full depth and part of a second. Two widths beneath the beam are this reimplementation’s own rather than the original’s: it offers the pruning prompt the first 40 relations of an entity and the first 20 neighbours of a relation, against the 300 and 200 AGR sees (S2.5). Reimplementing is a stronger comparison than citing the original figures, because graph, model, budget, and scoring rule are held constant and only the algorithm varies. It also means its numbers are not comparable to the published ones, and we do not present them as such. All baselines were debugged on development data and certified before any test question ran, against a pre-specified diagnostic targeting the one failure that would be an implementation defect rather than a paradigm limit: retrieval found the gold answer and the system hedged anyway.

S3.3. *How groundedness is measured*

Groundedness is measured at two levels. The structural tier is deterministic and applies to every asserted entity in all 4,000 records: an entity is grounded if and only if it exists as a node in the environment and connects to a topic entity within four hops. The semantic tier asks the harder question of whether the entity is supported *as the answer*. For a stratified sample of 60 answers per system per dataset, it retrieves the graph’s own shortest connecting paths and has a language model judge whether any path expresses the relationship the question asks about, under the governing instruction that mere connectedness is not support. Because path retrieval takes only the three shortest paths, semantic-tier rates are conservative lower bounds. They are depressed uniformly across systems and read as a between-system comparison rather than as absolute support rates. Section 4.3 reports the judge’s validation against a human annotator, including the threshold it missed.

S4. Secondary measurements

Five measurements are reported here rather than in the body. Each supports a headline the body states; none carries a conclusion of its own.

S4.1. Latency

Section 5.2 reports calls and tokens, both exact from the shared meter. Wall-clock time tells the same story in seconds: a mean of 11.2 and 17.2 per question for AGR against 12.9 and 15.2 for the baseline, and between 1.0 and 2.2 for the three single-call systems. Read those as magnitudes only. Records replayed entirely from a response cache are excluded from those means, but between 3.6% and 15.2% of the two navigators’ calls in the remaining records were still served from the cache, which shortens a cached call to nothing. The figures are therefore lower bounds and do not rank the two navigators against each other at the second.

S4.2. Positioning against published RoG results

The environment of Section 4.1 is built from the per-question subgraphs that the RoG distribution publishes, and RoG [5] is the closest published system in the path-retrieval family, so a reader who notices both will ask how the two compare. The aggregate answer does not favour this work, and we give it before the qualifications. Over the full test splits RoG publishes 85.7 Hits@1 and 70.8 F1 on WebQSP and 62.6 and 56.2 on ComplexWebQuestions, against this work’s 75.5 and 64.2, and 52.2 and 46.9 over the certified 400-question samples of those same splits. **RoG leads on all four figures, and the margin is not sampling noise:** AGR’s 95% intervals on Hits@1 are [71.3, 79.7] on WebQSP and [47.5, 57.0] on ComplexWebQuestions, and RoG’s 85.7 and 62.6 fall outside both.

One difference bounds what that comparison establishes, and it is decisive. **RoG is trained on these benchmarks; AGR is trained on nothing.** RoG fine-tunes LLaMA2-Chat-7B on the training splits of both benchmarks, which hold 2,826 and 27,639 questions in the distribution used here, while AGR’s backbone is frozen and prompted and has seen no question from either benchmark before evaluation. Three ordinary differences follow it: the backbones differ, which is the very factor Section 4.2 holds constant elsewhere; RoG reports over the full splits against our stratified samples of them and publishes no interval, so the gap can be bounded from one side only; and RoG retrieves within each question’s own published subgraph while AGR navigates the union of all of them. What the comparison settles is AGR’s place in the published landscape, a training-free system against a supervised one on the same question sets. What it does not settle is anything this paper measures, since neither the ablation of Section 6 nor the census of Section 7 has a counterpart in RoG’s evaluation.

S4.3. Accuracy by hop depth

Stratifying by the number of hops separating a topic entity from the gold answer separates the systems further. On ComplexWebQuestions, AGR is the only system whose accuracy *rises* with hop count: 0.46 at one hop, 0.55 at two, 0.57 at three or more, over strata of 137, 211, and 49 questions. The last step sits well inside the interval of a 49-question stratum, so the claim is the direction across the three strata, not the size of any one step. Every other system ends below where it started. That phrasing is deliberate and the weaker alternative is the true one: Think-on-Graph does not decay monotonically but falls from 0.53 to 0.37 and then partly recovers to 0.45. It also beats AGR on that dataset’s one-hop stratum, 0.53 against 0.46, and Section 6.1 identifies the mechanism, since

Table S1: Semantic-tier support rate: the share of sampled assertions a judge found supported *as the answer* by the graph’s own connecting paths. $n = 60$ per cell, so the 95% binomial interval has a half-width of roughly 12 to 13 points. These are conservative lower bounds by construction (Section 4.3).

System	WebQSP	CWQ
no-retrieval	63.3%	36.7%
Vector-RAG	61.7%	50.0%
GraphRAG	55.0%	36.7%
Think-on-Graph	61.7%	43.3%
AGR	66.7%	48.3%

decomposition costs accuracy on questions that do not need decomposing and that stratum is full of them.

The static graph baseline’s decay across these strata is deliberately *not* offered as evidence for the paradigm, because its retrieval radius confounds it, and the evidence that it is a radius effect is in the strata themselves. GraphRAG scores 0.44 on WebQSP’s two-hop stratum, where those questions are largely mediator paths a one-hop expansion does reach, against 0.16 on the ComplexWebQuestions two-hop stratum, where the chains are genuine compositions it cannot reach at all. A retriever failing uniformly on depth would not show that gap. A rising curve is what decomposition would predict, but it is consistent with that mechanism rather than evidence for it, and the WebQSP panel carries no comparable evidence: its three-hop stratum contains four questions, which is why the claim is scoped to one dataset.

S4.4. Semantic-tier support

Section 5.4 reports the structural tier. The semantic tier asks the harder question of whether an asserted entity is supported *as the answer*, and it separates the systems far less. Table S1 gives every cell rather than a selection, because the ordering differs by dataset and any subset of it misleads.

Every system lands in a 36.7–66.7% band. AGR is highest on WebQSP at 66.7%. **On ComplexWebQuestions it is second**, 5.0 points ahead of the agentic baseline but 1.7 behind Vector-RAG’s 50.0%, so its margin over Think-on-Graph is 5.0 points on both datasets and its lead over *every* system is a WebQSP result only. Neither gap survives the sample: at $n = 60$ per cell a 5-point difference sits well inside the interval, as does Vector-RAG’s 1.7, and no conclusion in this paper rests on either. What the tier establishes is the level rather than the ranking, and the level is the more useful finding, which Section 5.4 states.

S4.5. Consensus rows behind the cleared questions

Table S2 is the row-level backing for the reading Section 7.2 gives of the 64 questions adjudication cleared. A row is one consensus answer, and a question can carry more than one.

S4.6. The next experiments, and what each would settle

Section 8.4 names four follow-ups in the order they would be run. First, the agentic baseline at AGR’s candidate widths under a sweep of the call cap, with the budget split of Table 2 reported at each setting, which tests whether the affordability reading survives equal access to the graph. Second, replication: every system and every ablation condition run several times, so that a delta of a few hundredths of F1 can be read against the run-to-run variance rather than assumed to exceed it, together with the joint-removal condition Section 6.3 names. Third, the full test splits,

Table S2: The 64 flagged questions that adjudication cleared, read as consensus rows (one row per consensus answer). The triage family is the pre-pass’s own label for why the systems agreed on a wrong entity. The lower rows count which systems were among those agreeing.

	WebQSP	CWQ
Consensus rows cleared as not a label defect	54	31
filed as echo attractor	36	16
filed as relation confusion	16	3
filed as constraint ignored	0	12
filed as environment gap	2	0
Rows on which the no-retrieval control agrees	15	25
Rows on which both navigators agree	26	14
Echo rows on which the no-retrieval control agrees	13	11
Echo rows on which both navigators agree	20	9

Table S3: The budget split of Table 2 read within hop strata: n and Hits@1 for the two navigators on the questions the baseline finished and the questions the cap clipped. The four three-hop WebQSP questions and the unreachable strata (12 and 3 questions) are omitted.

Dataset	Hops	Think-on-Graph finished			Think-on-Graph clipped		
		n	Think-on-Graph	AGR	n	Think-on-Graph	AGR
WebQSP	1	175	0.846	0.766	81	0.160	0.728
WebQSP	2	100	0.910	0.890	28	0.321	0.679
CWQ	1	69	0.754	0.609	68	0.309	0.309
CWQ	2	123	0.561	0.610	88	0.102	0.477
CWQ	3+	31	0.613	0.581	18	0.167	0.556

which quadruple the discordant pairs the paired tests see. Fourth, one logging change: persist each accepted claim with the route that accepted it and the triples that matched, which makes the verifier’s precision and recall a number a labelled sample can produce.

S4.7. The budget split within hop strata

Section 5.3 reads the budget split of Table 2 within hop strata, to check that the clipped half is not simply the hard half, and quotes every figure it argues from. Table S3 gives the full breakdown.

S4.8. The failure census by category

Figure S2 is the distribution behind Section 7, which names its largest categories in prose and argues from two of the smaller ones.

S4.9. What bounds the two larger nulls

Section 6.2 reports that backtracking and model scoring change nothing detectable. Two implementation facts bound those nulls further, and we record them rather than smooth them over.

The backtracker bans the edges of the most recent explorer pass but restores the highest-scoring snapshot, which is often older, so everything expanded in between stays unbanned (S2.3). Over the main runs, 53 explorer passes re-expanded an anchor-and-relation set the same question had already expanded, and at least 65 of the 262 backtracks restored a depth other than the most recent snapshot’s. The null therefore describes backtracking as implemented, not backtracking as specified.

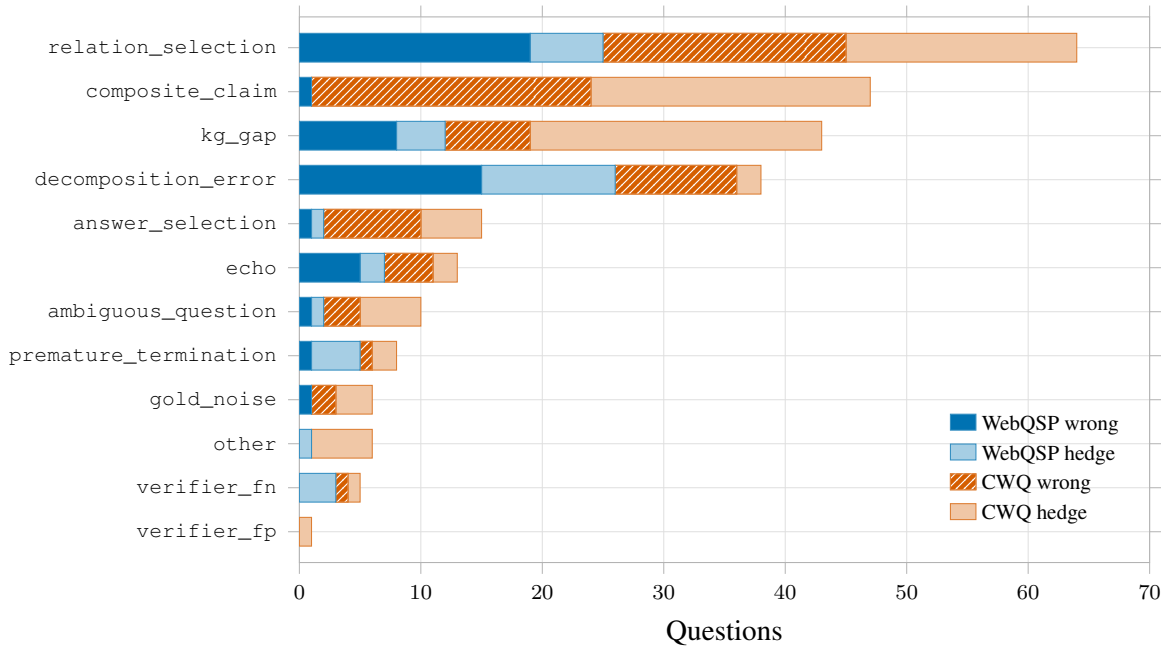


Figure S2: Hand-read census of the 256 residual failures read, by category, with wrong answers and hedges kept separate. Relation selection and composite claims dominate. The two categories Section 7 discusses are not the largest, but they are the two whose lessons transfer beyond this system.

The development-set sweep that froze α points the same way as the scoring null at a smaller scale: at $\tau = 0.20$ it scored 61, 62, 64, and 59 hits of 80 at $\alpha = 0.3, 0.5, 0.7$, and 1.0, so dropping the model term cost five hits there, before the entity-filter correction that preceded the test runs. One confound goes with it: at $\alpha = 1$ the scorer takes an early return that leaves the backtracking trigger’s signal maximum computed over a smaller candidate set, so that condition’s elevated backtrack count is not attributable to the scoring change alone.

S4.10. What an equal-width re-run would and would not settle

Section 5.3 reports that the agentic baseline searched a narrower candidate pool than AGR, so its figures on the questions it finishes are a lower bound rather than an estimate, and that equalising the widths is the first change we would make in a follow-up. That re-run would not vary one thing, though. Wider candidate sets make each pruning call dearer, so equal-width pruning would also raise the baseline’s clip rate, moving the boundary between the two halves of Table 2 at the same time as it changes what the search resolves within each. The two effects have to be read apart, which means reporting that split again at each setting rather than an aggregate in which they cancel or compound invisibly.

A related check bounds the aggregate from the other side, because a metric that scores a hit on any match rewards naming more entities. On the WebQSP questions it answers, AGR names 3.37 entities against Think-on-Graph’s 2.56, and its entity precision over those same answered questions is 0.760 against 0.703; on ComplexWebQuestions the figures are 1.54 against 1.40 entities and 0.629 against 0.520 precision. If AGR bought breadth without accuracy, precision would fall as the entity count rose. It does not fall.

S4.11. Which systems joined each cleared consensus

Section 7.2 reports that the systems agreeing on a cleared consensus answer are not independent observers. These are the counts behind that. On ComplexWebQuestions the no-retrieval control, which sees no graph at all, is among the agreeing systems on 25 of the 31 cleared rows and on 11 of the 16 echo rows, so there the convergence is at least as much a property of what the backbone believes as of the graph’s neighbourhood structure. On WebQSP the control joins 15 of 54 rows and 13 of 36 echo rows, and the two navigators agree with each other on 20 of the 36. Of the 13 rows on which all five systems agreed, 9 were cleared as not a defect.

S4.12. The verifier’s unmeasured exposure, in claims

Section 8.1 names wrongful acceptance as the most serious gap in this work and says why the committed record cannot close it. This is its scale in the claim unit. There is no way, from that record, to know how many wrongful acceptances sit inside the 761 questions that ended grounded. On test the verifier decomposed 2,110 claims and accepted 2,008 of them; the graph-wide adjacency probe accounts for 39 of those acceptances, and the rest were made either by traversed adjacency, which is equally relation-blind, or by the entailment check, which is not, and the record does not say which. So the share of accepted claims certified without any test of the asserted relation lies somewhere in [39, 2,008]. That interval is two orders of magnitude wide, and we report it as one rather than choose a point inside it. The same gap means the layer has no measured precision or recall as a classifier of claims, which is the first number a verification component should carry.

The output contract is bounded by the same logging decision. On test, 12 of AGR’s 367 answered WebQSP questions and 9 of its 308 on ComplexWebQuestions were emitted with a count of zero supporting triples, which is the contract’s own measure of how often it delivers nothing.

S4.13. Sensitivity of the tables to the exclusions

Section 7.2 reports that the accuracy tables are not rescored against the adjudicated labels. This is how much that decision could matter. The defects cut both ways: 10 of the 22 WebQSP exclusions and 4 of the 19 on ComplexWebQuestions are questions AGR scored as hits against a label now judged defective. Recomputing every cell of Table 1 with the adjudicated questions removed moves each system by between +0.001 and +0.020 Hits@1 and leaves the ranking on both datasets unchanged. That shift is an order of magnitude below the 5.5% and 4.8% defect rates, because removing a question the system got right subtracts from the numerator as well as the denominator. The exclusions therefore do not threaten any comparison this paper draws. What they establish is that a non-trivial share of both benchmarks is mislabelled, which is a fact about the benchmarks rather than a correction to be applied to them.

References

- [1] N. Reimers, I. Gurevych, Sentence-BERT: Sentence embeddings using Siamese BERT-networks, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, 2019, pp. 3982–3992.
- [2] L. Chen, P. Tong, Z. Jin, Y. Sun, J. Ye, H. Xiong, Plan-on-Graph: Self-correcting adaptive planning of large language model on knowledge graphs, in: Advances in Neural Information Processing Systems, Vol. 37, 2024, arXiv:2410.23875.

- [3] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, J. Larson, From local to global: A graph RAG approach to query-focused summarization, arXiv preprint arXiv:2404.16130 (2024).
- [4] J. Sun, C. Xu, L. Tang, S. Wang, C. Lin, Y. Gong, L. M. Ni, H.-Y. Shum, J. Guo, Think-on-Graph: Deep and responsible reasoning of large language model on knowledge graph, in: International Conference on Learning Representations (ICLR), 2024, arXiv:2307.07697.
- [5] L. Luo, Y.-F. Li, G. Haffari, S. Pan, Reasoning on graphs: Faithful and interpretable large language model reasoning, in: International Conference on Learning Representations (ICLR), 2024, arXiv:2310.01061.